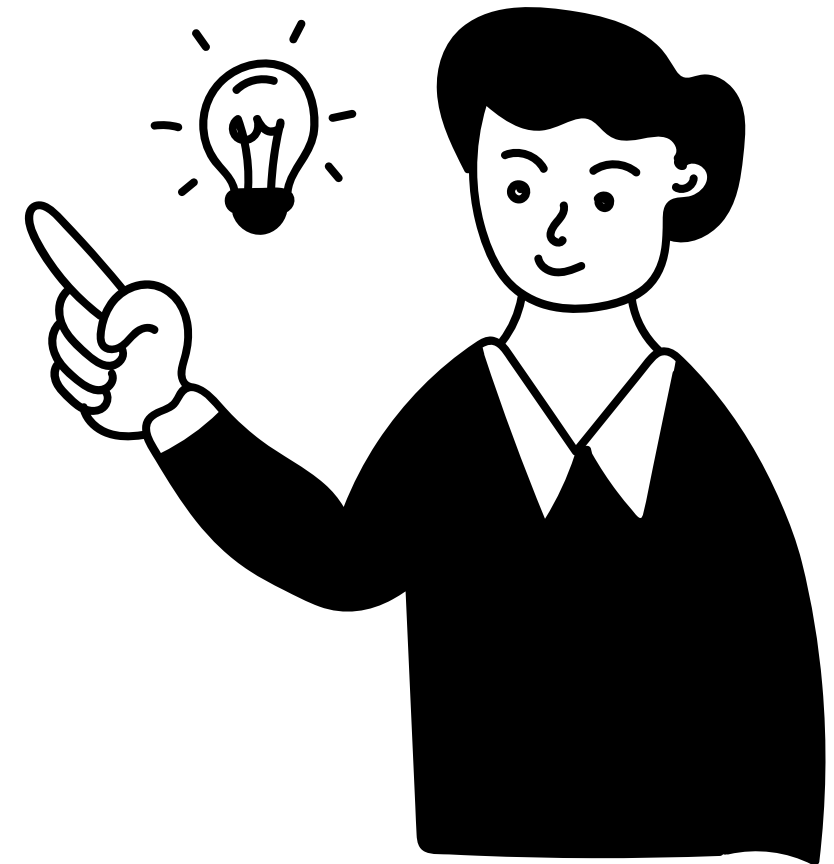


연말정산 세금 Q&A

RAG 시스템

6조 이수홍, 이수민, 이희영



목차

01	개요 / 역할	07	성능 개선 방안 5 : 검색 방식 및 top_k 최적화
02	베이스라인 구성 / 개선	08	성능 개선 방안 6 : LLM 모델 비교
03	성능 개선 방안 1 : 페이지 머리말 제거	09	성능 개선 방안 7 : Prompt 비교
04	성능 개선 방안 2 : 검색 정답 평가 기준 재정의	10	성능 개선 방안 8 : 평가셋 13→26문항 확대
05	성능 개선 방안 3 : 임베딩 모델 4종 비교 및 선정	11	성능 개선 방안 9 : Reranker 도입
06	성능 개선 방안 4 : 검색 결과 페이지 중복 제거	12	시연 / 결론 / 어려웠던 점

개요

01

프로젝트 목표

국세청의 연말정산 안내 PDF를 기반으로,
문서 전처리, 청킹, 임베딩, 검색 방식, LLM과
프롬프트 조합을 비교

사용자 질문에 정답과 근거를 제공하는
LangChain 기반 RAG 시스템을 구현

02

선정 이유

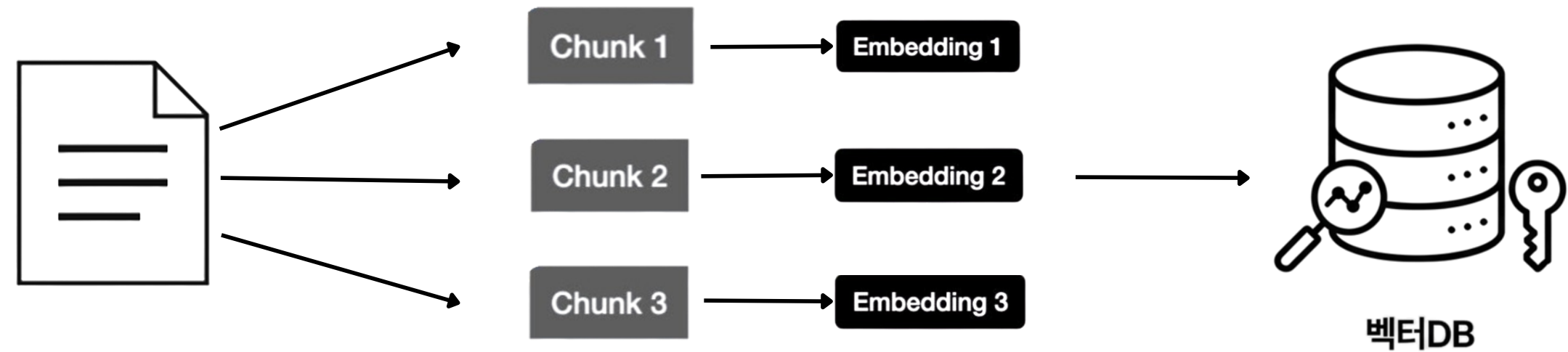
여러 전문 용어와 표로 구성되어 있지만, 사용자는
이를 일상적인 표현으로 질문하기 때문에
키워드 검색만으로는 정확한 정보를 찾기 어려움

그러나 공제율·한도·대상 조건처럼 **정답이 명확해**
객관적 성능 평가가 가능함

역할

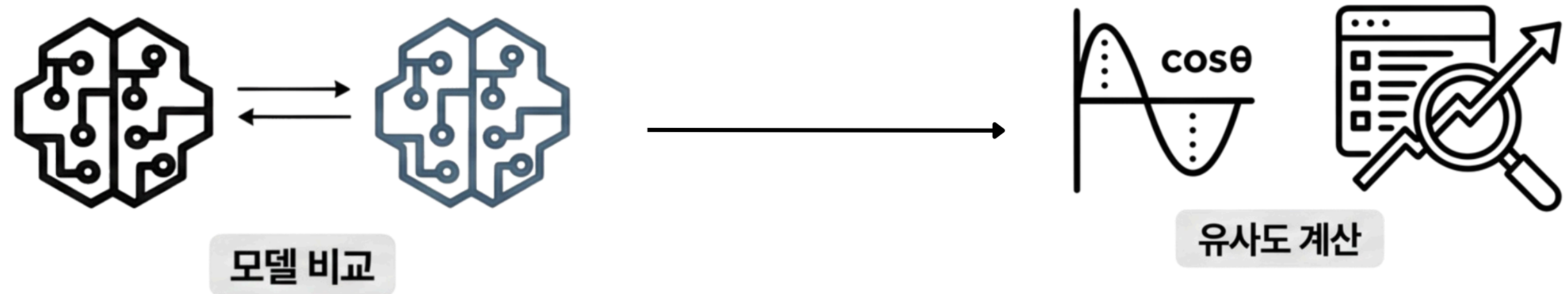
문서 분할(칭킹), 표 전처리,
벡터스토어 구축

<이수민>



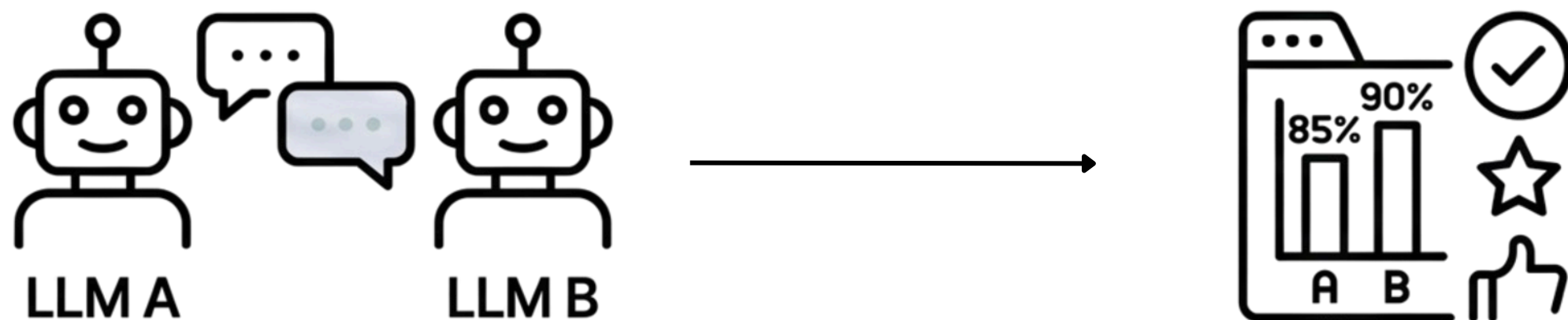
임베딩 모델 비교,
유사도 검색 알고리즘 향상

<이희영>



LLM 모델 비교 및
Evaluation 모델 작성

<이수홍>



베이스라인 모델 구성

항목	값
chunk_size / overlap / top_k	1000 / 200 / 3
평가 질문	13문항
벡터스토어	Chroma
임베딩	HuggingFace `jhgan/ko-sroberta-multitask`
LLM / 검색 방식	Gemini / Similarity
생성 청크 수	635

Hit@3	0.154
MRR	0.115
BERTScore F1	0.6961
평균 응답시간	6.56초

13문항 중 2문항만 정답 페이지 검색 성공

chunk_size 1000 / overlap 200
=> 큰 청크를 세분화하고 중복 텍스트를 줄여 개선

<평가 지표>
Hit@3 - 상위 3개 검색 결과에 정답 페이지가 포함된 비율
MRR - 정답 페이지가 검색 결과의 앞 순위에 나타나는 정도
BERTScore F1 - 생성 답변과 기준 답변 사이의 의미적 유사도
평균 응답시간 - 검색부터 최종 답변 생성까지 걸린 평균 시간

베이스라인 모델 개선

항목	값
chunk_size / overlap / top_k	500 / 100 / 3
평가 질문	13문항
벡터스토어	Chroma
임베딩	HuggingFace `jhgan/ko-sroberta-multitask`
LLM / 검색 방식	Gemini / Similarity
생성 청크 수	1132

Hit@3	0.385
MRR	0.192
BERTScore F1	0.7423
평균 응답시간	7.21 초

13문항 중 8문항에서 정답 페이지 검색 실패

< 베이스라인의 문제점 >
모든 페이지에 반복되는 머리말과 인쇄 페이지 번호가 청크에 포함
반복 문구로 인해 서로 다른 페이지의 의미적 차이가 약해짐



반복 머리말·인쇄 페이지 번호 제거를 통한
검색 노이즈 감소

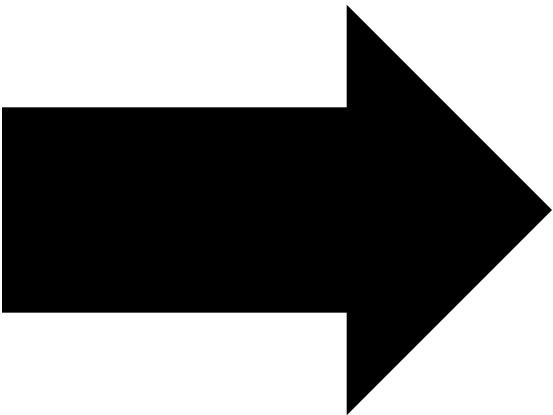
성능 개선 방안 1 : 페이지 머리말 제거

< 제거 전 >

원천징수의무자를 위한
2025년 연말정산 신고안내
202

○ 학교법인이 사립학교에 시설비·교육비...
(기부금 관련 조항 — 생략)...
09 월세액 세액공제(조특법 §95의2)
무주택 세대의 세대주로서 총급여액이
8천만원 이하인 근로소득이 있는 거주자가
...월세액(연 1,000만원 한도)의
15%(17%)에 해당하는 금액을 공제한다.

제거된 항목
반복 머리말 2줄 + 인쇄 페이지번호(202)



< 제거 후 >

○ 학교법인이 사립학교에 시설비·교육비...
(기부금 관련 조항 — 생략)...
09 월세액 세액공제(조특법 §95의2)
무주택 세대의 세대주로서 총급여액이
8천만원 이하인 근로소득이 있는 거주자가
...월세액(연 1,000만원 한도)의
15%(17%)에 해당하는 금액을 공제한다.

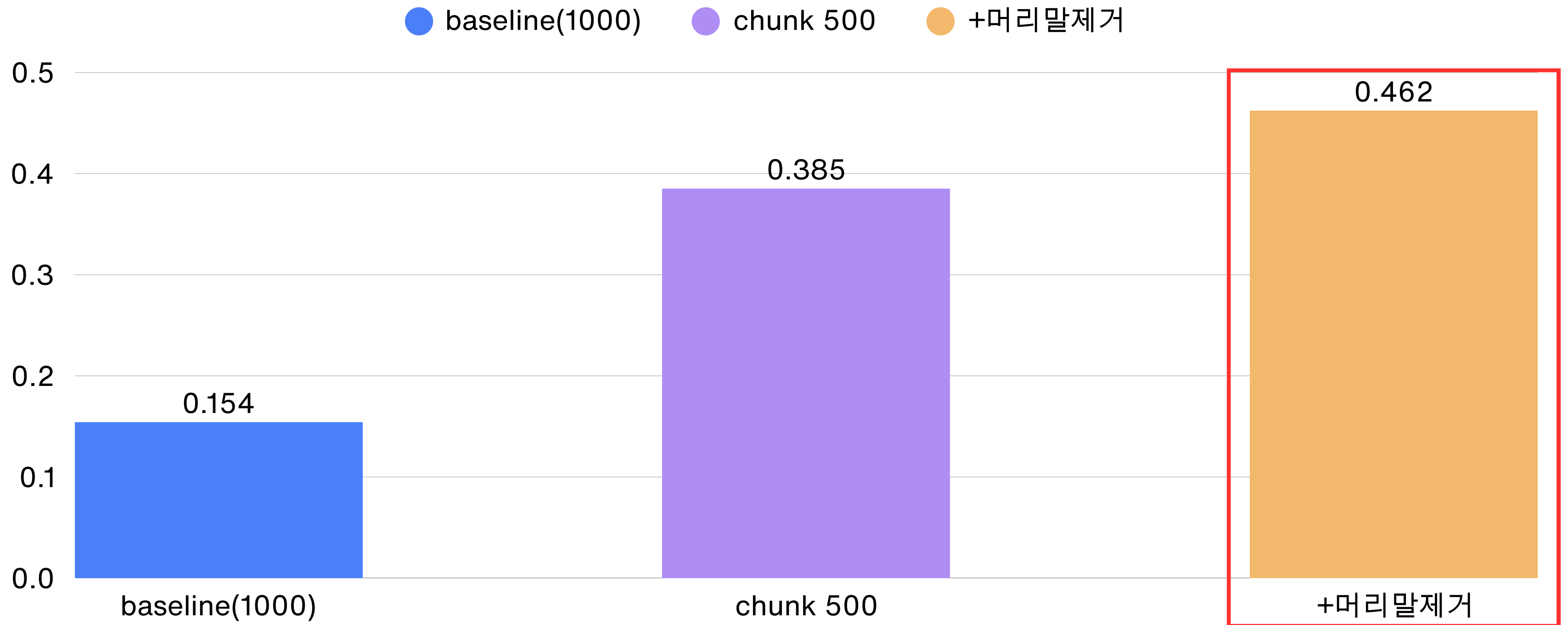
Hit@3

0.385 → 0.462

13문항 중 7문항이 여전히 검색 실패로 기록됨
=> 기존에는 정답 페이지를 하나만 등록했기 때문에 실패로 처리

※ 문서처리 파이프라인 단계별 개선 - 정답지 하나

13문항 - Hit@3



단일 정답 기준에서는 청킹·머리말 제거를 거치며 Hit@3가 0.154→0.462로 향상

성능 개선 방안 2 : 검색 정답 평가 기준 재정의

Q3. 8세 이상 자녀가 2명이면
자녀세액공제를 얼마나 받나요?

기존

기존 정답 페이지
p.6

검색 결과
p.182 → p.20 → p.59

판정: 검색 실패

기존 정답표는 대표 페이지 하나만
정답으로 지정

실제 정답 내용이 있는 상세 페이지를
검색해도 실패로 처리

13문항 중 Q10 1문항만이
검색 실패로 기록됨

개선

복수 정답 페이지
p.6, **p.182**

동일한 검색 결과
p.182 → p.20 → p.59

판정: 검색 성공

p.182 실제 근거
"8세 이상 자녀가 2명인 경우 세액공제액은 55만원"

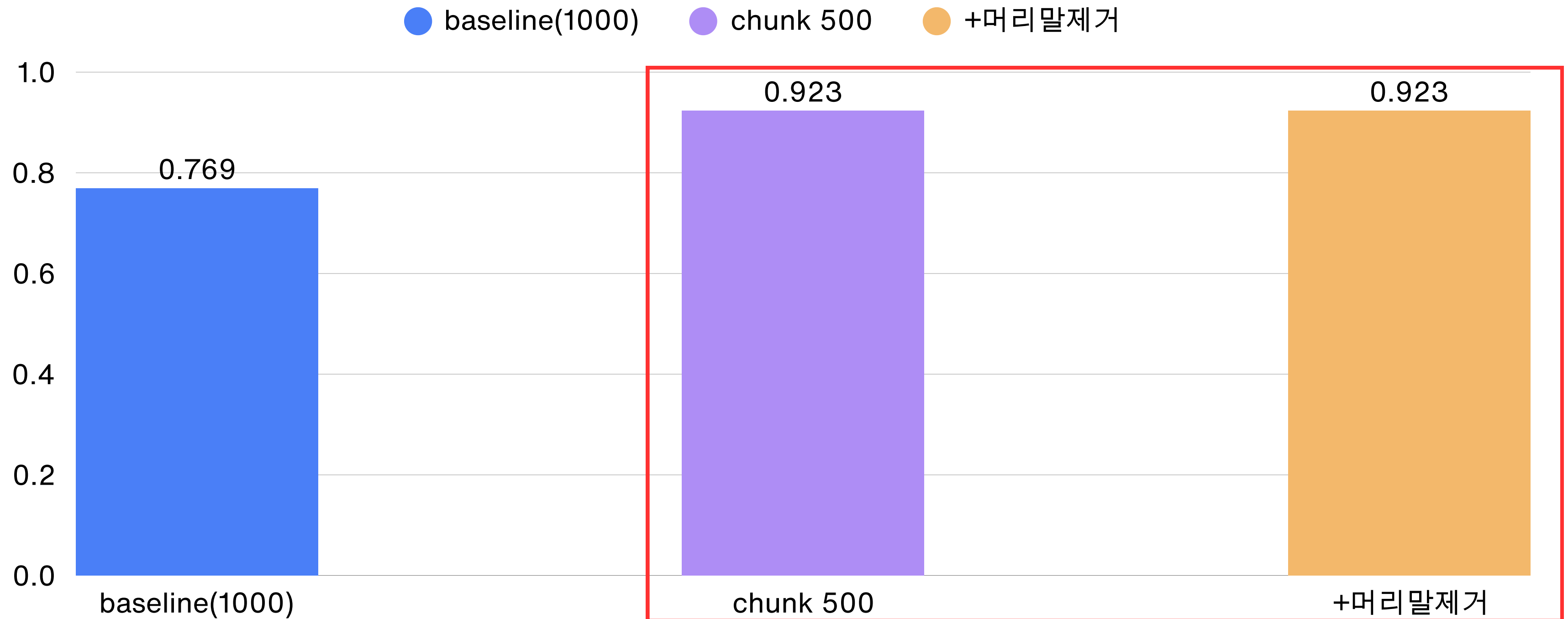
질문별로 정답이 존재하는
모든 페이지 조사

검색된 상위 3개 중 **하나라도**
정답 페이지와 일치하면 성공 처리

Hit@3	0.462 → 0.923
MRR	0.244 → 0.808

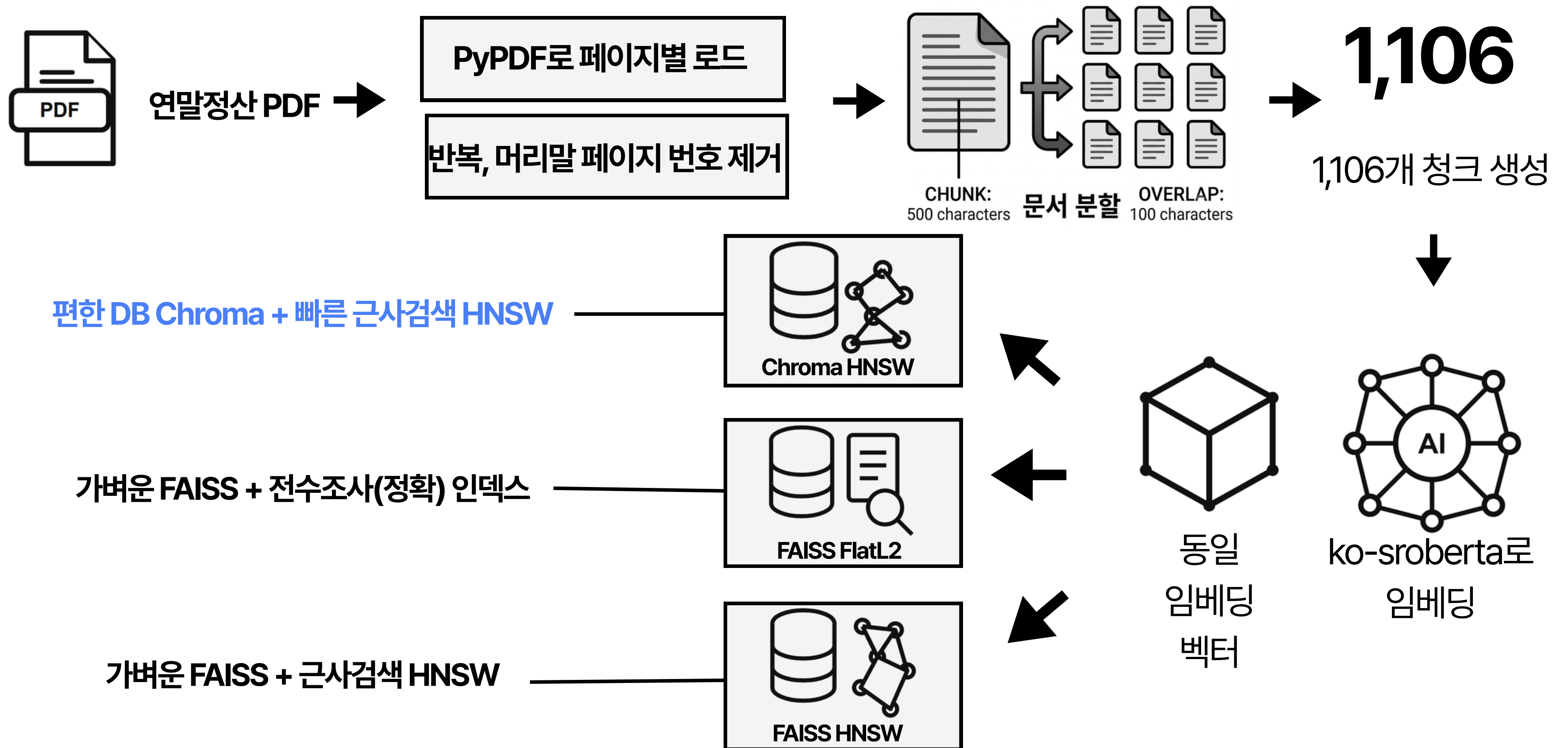
※ 문서처리 파이프라인 단계별 개선 - 정답지 여러 개 인정

13문항 - Hit@3

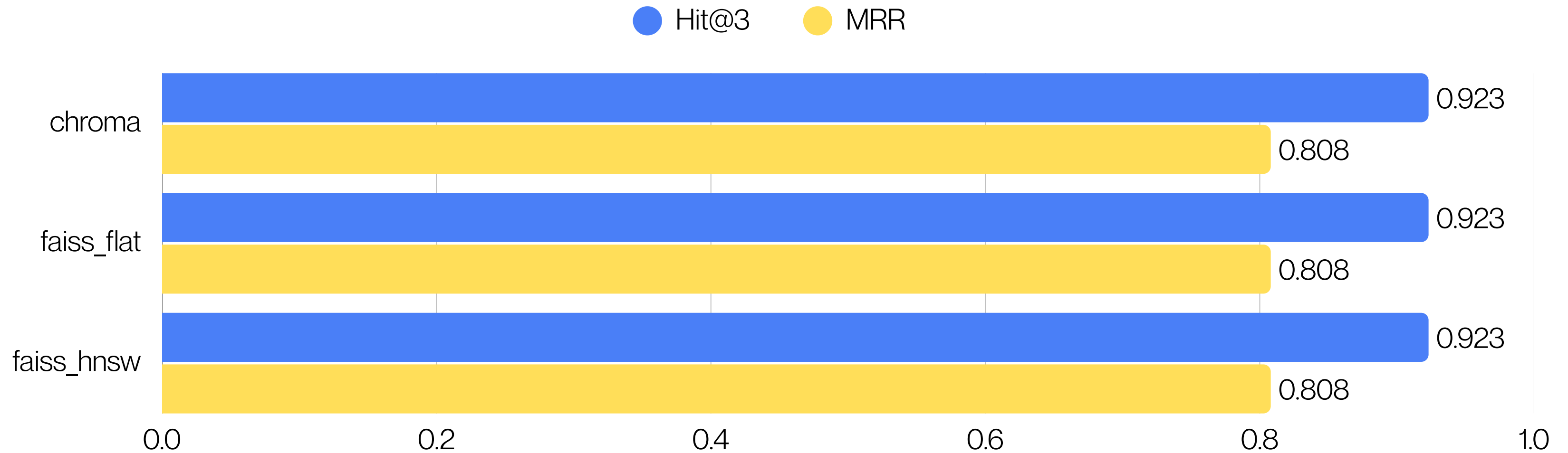


청킹 파라미터 조정으로 Hit@3가 0.769→0.923으로 향상됐으며, 머리말 제거는 Hit@3를 유지하면서 **MRR을 0.731→0.808로 개선**

※ 벡터스토어 구축



※ 벡터스토어 다중 비교 - chroma로 최종 채택한 이유



점수가 같은 이유?

1. 같은 임베딩 벡터를 사용했다.
2. 문서가 1,106개로 적어서 세 방식 모두 같은 가까운 문서를 찾았다.
3. 조금 달라진 오답 순서는 Hit@3와 MRR에 영향을 주지 않았다.

Chroma 최종 채택 이유

1. 세 방식의 검색 정확도에 차이가 없었다.
2. 기존 프로젝트 코드와 바로 연동할 수 있었다.
3. 벡터 저장·재사용과 페이지 정보 관리가 편리했다.

성능 개선 방안 3 : 임베딩 모델 4종 비교 및 선정

모델	Hit@3	MRR
jhgan/ko-sroberta-multitask	0.923	0.808
intfloat/multilingual-e5-small	0.692	0.615
intfloat/multilingual-e5-base	0.846	0.577
BAAI/bge-m3	0.923	0.923

※ 비교 대상: 한국어 특화 1종과 다국어 3종 (전부 로컬 HF 모델)

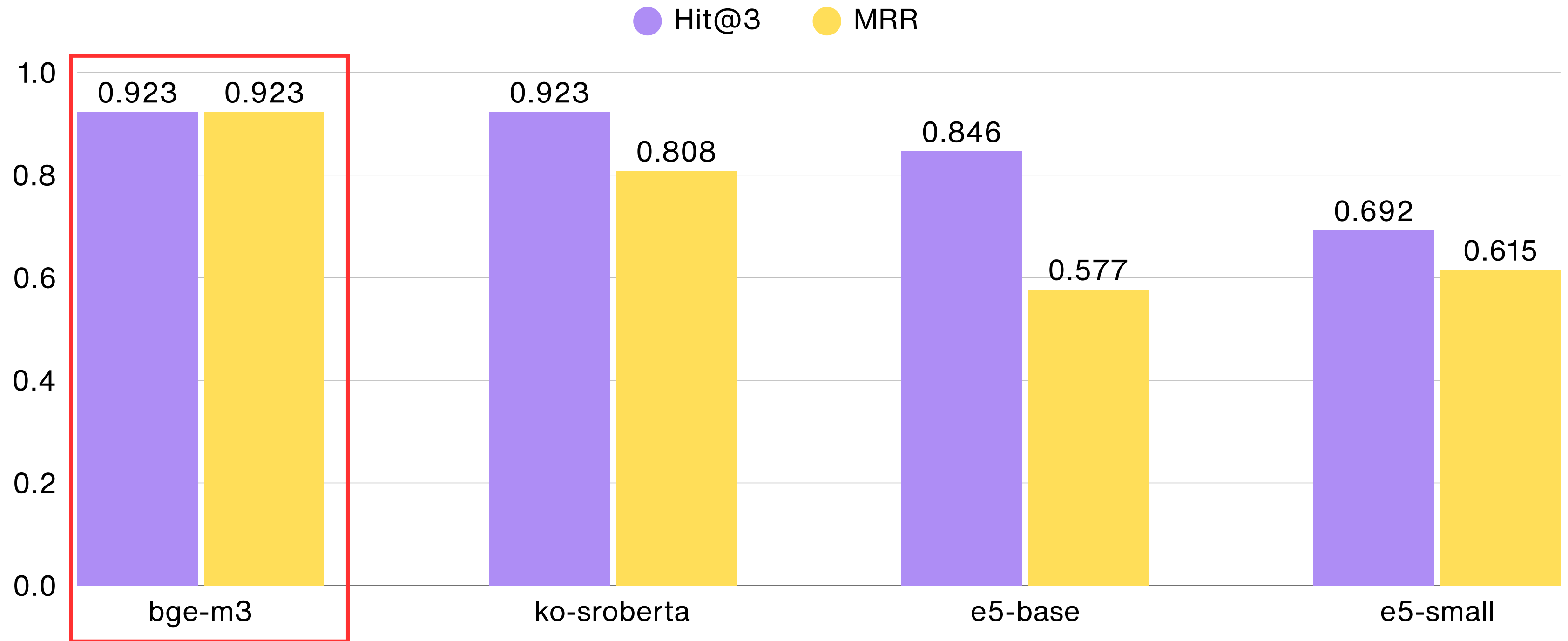
<조건>
청킹·전처리 고정 상태에서
임베딩 모델만 변경해 검색
품질 비교

비교 모델 선정 기준
ko-sroberta: 한국어 특화·경량 베이스라인
e5-small/base: 다국어 모델의 크기별 성능 비교
bge-m3: 일상어와 세무 용어 간 의미 검색 후보
전 모델 로컬 실행·API 비용 0원

Hit@3가 같아도 **정답 페이지를 더 높은 순위에 배치한
bge-m3를 검색 평가 1순위로 선정**

※ 임베딩 모델 비교

13문항 - Hit@3/MRR



bge-m3는 ko-sroberta와 Hit@3는 같았지만 MRR이 0.923으로 가장 높아 **정답 페이지를 더 상위에 배치**

성능 개선 방안 4 : 검색 결과 페이지 중복 제거 (dedup)

Q8. 부양가족 기본공제는 1명당 얼마이고 소득 요건은 어떻게 되나요?



bge-m3 적용 후 Q8은
동일한 페이지의 청크가 Top-3
를 독점하는 문제

	Hit@3	MRR	실패 문항
중복 제거 전	0.923	0.923	Q8
중복 제거 후	1.000	0.962	없음

13개 문항에서는 Hit@3이 1.000을 달성
그러나 단일 설정만으로 검색 방식이
최적인지는 확인되지 않음
→ top_k, Similarity·MMR·Hybrid
검색 방식 비교

성능 개선 방안 5 : 검색 방식 및 top_k 최적화

검색 방식	Hit@3	MRR
Similarity	1.000	0.962
MMR λ 0.2→0.8	0.923	0.923 → 0.923
hybrid BM25 0.2→0.8	0.923	0.923 → 0.346

top_k	Hit@3	MRR
1	0.923	0.923
2	1.000	0.962
3~10	1.000	0.962

Similarity

의미 유사도 순으로 검색한 뒤 동일 페이지 제거

MMR

질문 관련성과 검색 결과의 다양성을 함께 고려

Hybrid

임베딩 의미 검색과 BM25 키워드 검색 결합

MMR - Q8을 해결하지 못함

Hybrid - 키워드 비중이 커질수록 검색 순위가 하락함

→ 따라서 의미 검색에 페이지 중복 제거를 적용한 Similarity로 결정

top_k는 검색할 때 몇 개의 청크를 가져올지 정하는 숫자

→ 검색 포화점 2에 안전 마진을 더해 3으로 결정

성능 개선 방안 6 : LLM 모델 비교

실험 조건
bge-m3 · Similarity · Page Dedup
fetch_k=15 · top_k=3 · Basic Prompt
13문항 · 동일 검색 문맥 사용

LLM	숫자 정확도	조건 포함률	BERTScore F1	평균 응답시간	잘못된 거절
Gemini	0.821	0.808	0.720	6.58초	1
OpenAI-mini	0.872	0.769	0.729	2.04초	0
Upstage solar-pro	0.872	0.885	0.703	2.30초	0
Qwen2.5-1.5B	0.769	0.641	0.702	47.44초	0
EXAONE-3.5-2.4B	0.744	0.821	0.697	83.84초	0

모델별 결과
Gemini: 응답시간이 가장 긴 API 모델이며 Q10을 잘못 거절
OpenAI: 숫자 정확도·BERTScore 우수, 조건 포함률은 상대적으로 낮음
Upstage: 숫자 정확도 공동 1위, 조건 포함률 전체 1위
Qwen: 로컬 모델 중 숫자 정확도가 높지만 조건 누락과 긴 응답시간 발생
EXAONE: Qwen보다 조건 포함률은 높지만 응답시간이 83.84초로 가장 김

최종 LLM 후보: Upstage solar-pro
숫자 정확도: 0.872 · 공동 1위
조건 포함률: 0.885 · 전체 1위
세무 답변에서 중요한 숫자나 여러 필수 조건을 종합하여 선정

성능 개선 방안 7 : Prompt 비교

Prompt	숫자 정확도	조건 포함률	BERTScore F1	평균 응답시간
Basic	0.853	0.904	0.712	2.10초
Simple	0.898	0.923	0.681	3.40초
Reasoning	0.838	0.923	0.744	1.60초

프롬프트 차이

Basic 문맥만 사용·숫자 정확히 인용·근거 페이지 표시·문서에 없으면 거절

Simple "문맥을 참고하여 질문에 답하세요"만 제시

Reasoning 공제항목 파악 → 요건·공제율·한도 확인 → 문맥만으로 답변 → 최종 답변만 출력

최종 Prompt 선택

숫자 정확도만 보면 Simple이 가장 높았지만,
거절 안전성·답변 형식·의미 유사도·응답시간
을 종합해 Reasoning Prompt를 선정

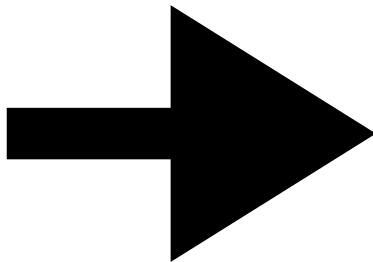
13문항의 초기 최종안 선정

bge-m3 + Similarity + Dedup + top_k=3 + Upstage + Reasoning Prompt

성능 개선 방안 8 : 평가셋 13→26문항 확대

13문항에서는 검색 만점이었지만
26문항에서는 Hit@3가 약 0.870으로 하락
조건 포함률도 0.923 → 0.696으로 하락

표·일상어 질문·문서 밖 질문에서 실패 발생



Rewrite 도입
“연금 얼마 돌려받나요?”
일상어와 문서의 공식 제도명 사이
의미 차이를 줄이기 위해 도입함

**정답 페이지 자체가 top-k 후보에 안 들어오는
문제 질문을 문서 용어로 다시 작성함**
→ 두 검색 결과의 순위를 합쳐서 최종 후보 선정

항목	Hit@3	MRR	실패 문항
질문 재작성 전	0.870	0.826	Q14, Q18, Q19
질문 재작성 후	0.957	0.833	Q18

성능 개선 방안 9: Reranker 도입

평가질문 26→66문항 확대

CHUNK_SIZE 500 → 300

66문항 확장 후 조합 재검증

Reranker 도입

검색 결과에 정답 문서가 있어도
순위가 낮아 Top-3에서 제외되는 문제 발생
→ 질문과 후보 문서를 다시 비교해
정답 가능성이 높은 문서를 상위로 재정렬

BAAI/bge-reranker-v2-m3

로컬 실행 · API 비용 없음 · 정확도 향상
※ 문서를 한 번 더 비교하므로 응답 시간은 증가

조합	Hit@3	MRR
dedup(기본값)	0.883	0.680
Rerank	0.917	0.794
Rewrite + Rerank	0.900	0.778



연말정산 Q&A

「2025년 연말정산 신고안내」를 학습한 AI에게 물어보세요.

>  실행 준비 상태

답변 모드



 빠른 답변

즉시 응답



 정밀 답변

재정렬로 더 정확 (몇 초 더)

예시 질문 — 눌러보세요

월세 살면 얼마나 돌려받나요?

8세 이상 자녀 2명이면 세액공제
얼마?

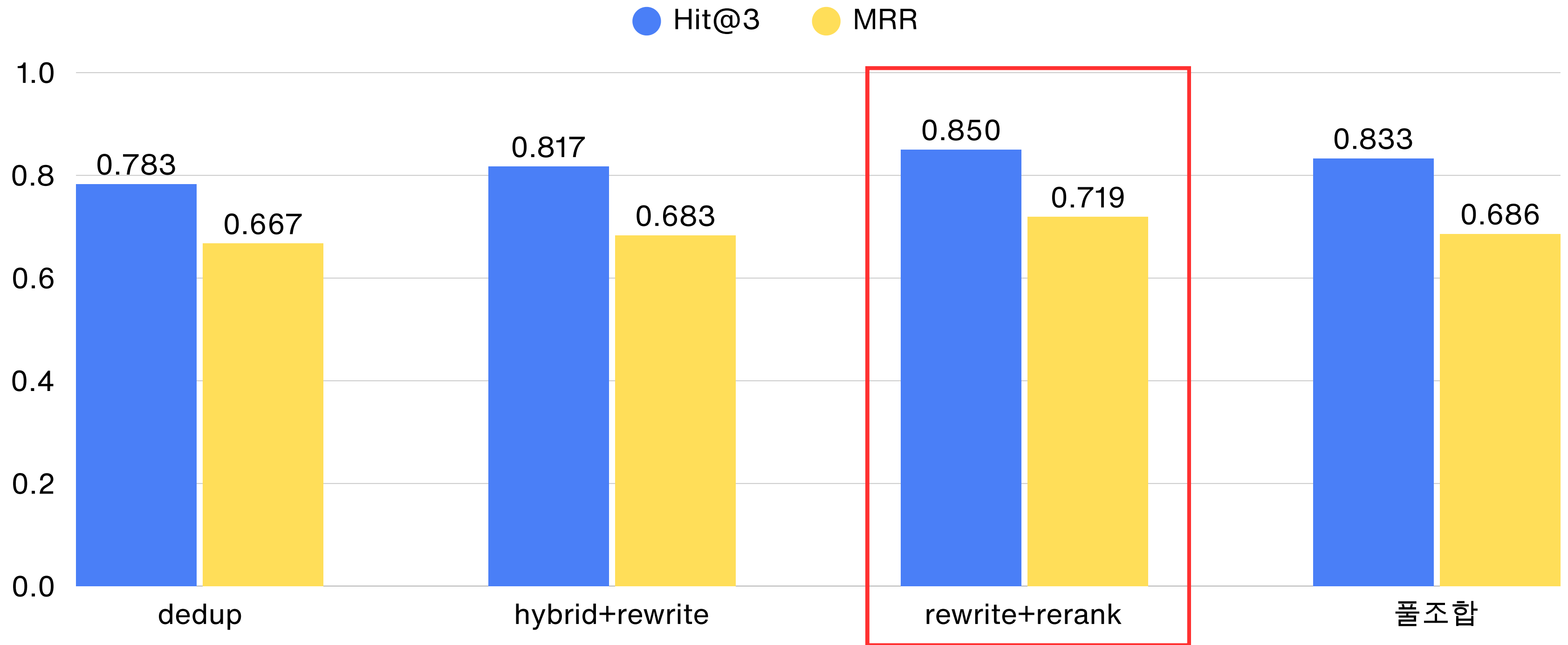
산후조리원 비용도 공제되나요?

예: 신용카드 소득공제는 어떻게 계산하나요?

질문하기

※검색 파이프라인 조합 (chunk 500)

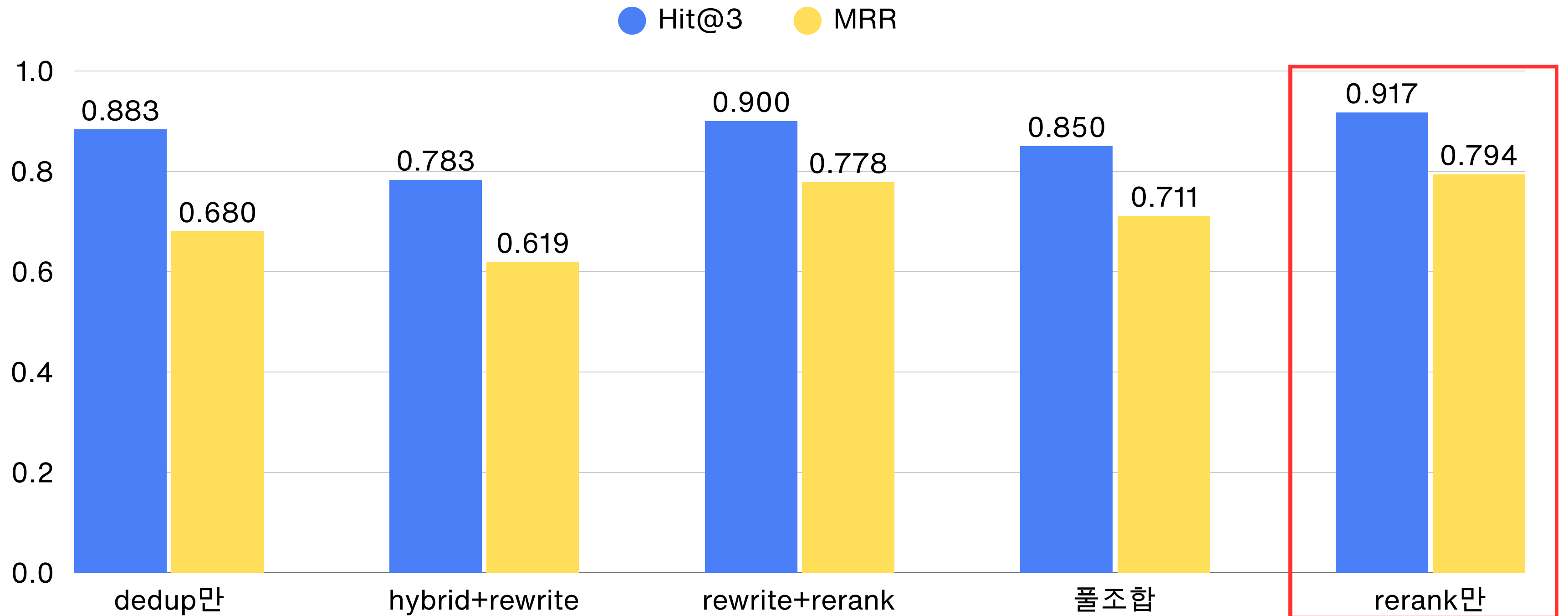
66문항 - Hit@3 / MRR



chunk500에서는 Rewrite+Rerank가 조합 중 가장 높은 검색 성능을 보임

※검색 파이프라인 조합 (chunk 300)

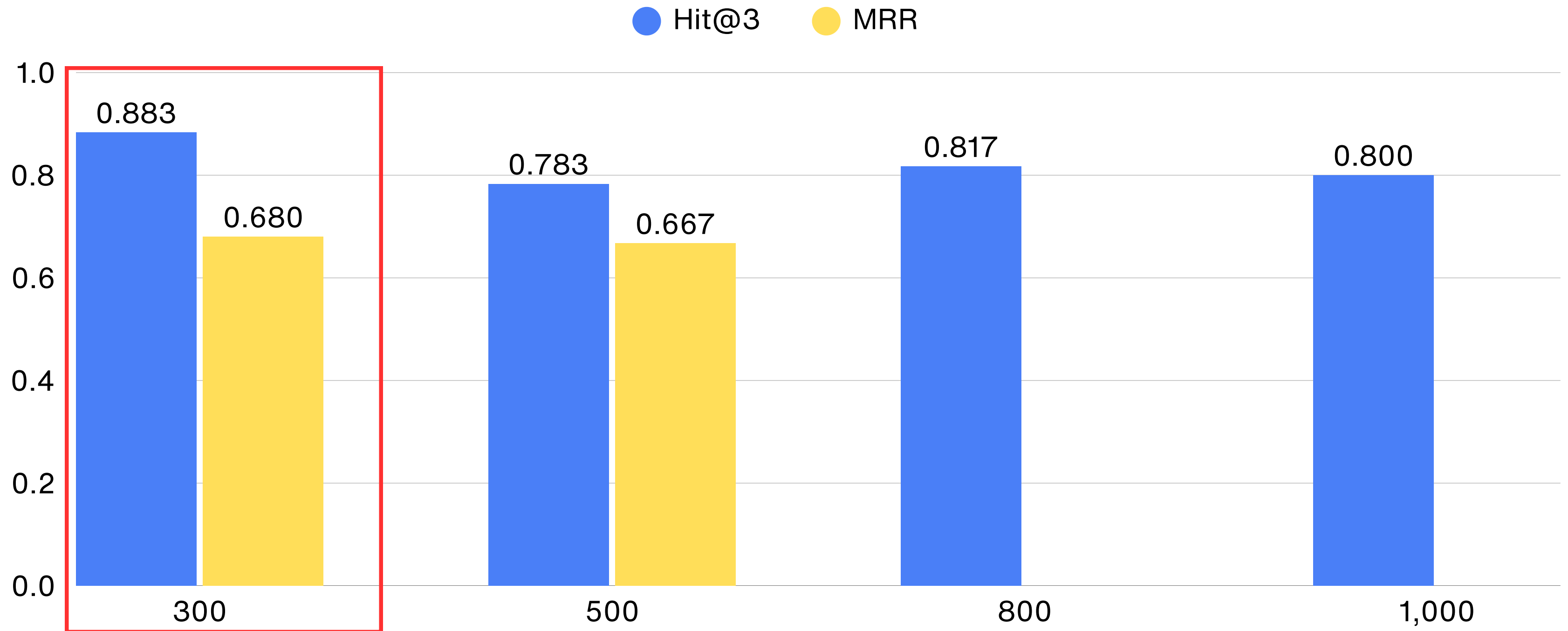
66문항 - Hit@3 / MRR



chunk300에서는 Rerank 후 Page Dedup을 적용한 구성이 가장 높은 검색 성능을 기록

Chunk 크기 재튜닝

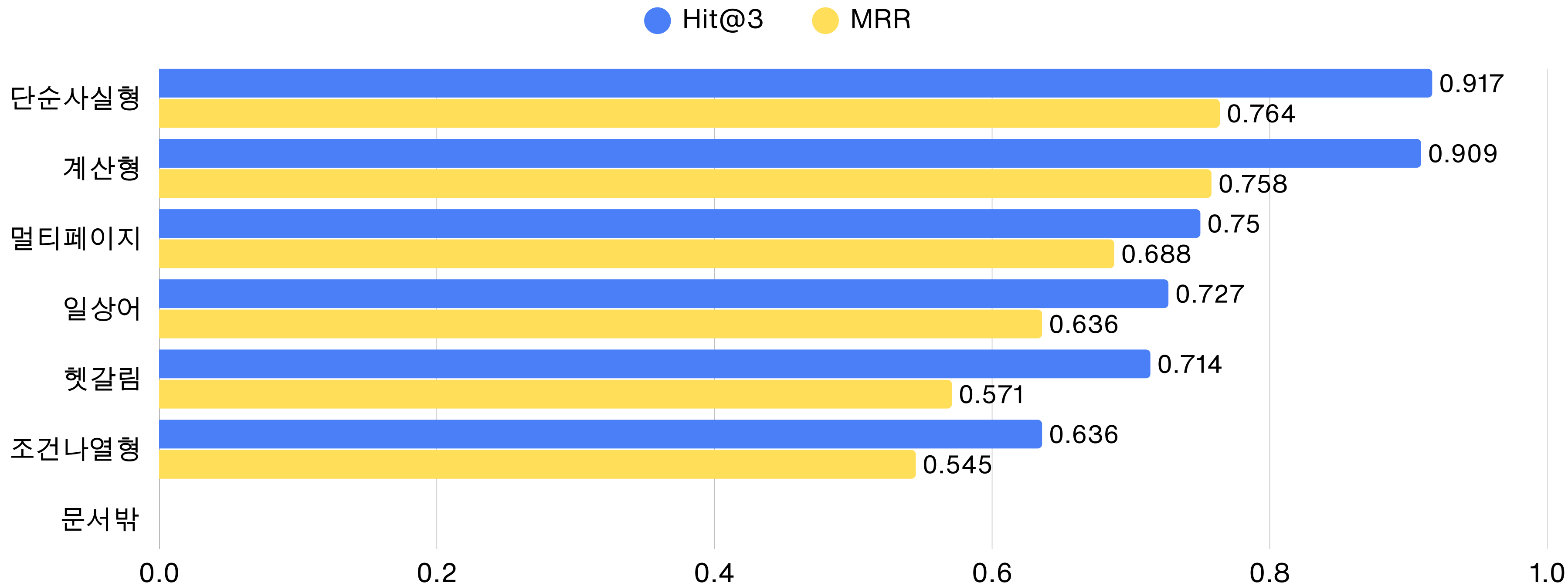
66문항 - Hit@3 / MRR (MRR은 300, 500만 측정)



chunk300이 두 검색 지표 모두 높아 **최종 청크 크기**로 채택

질문 유형별 성능 (기준선: chunk 500)

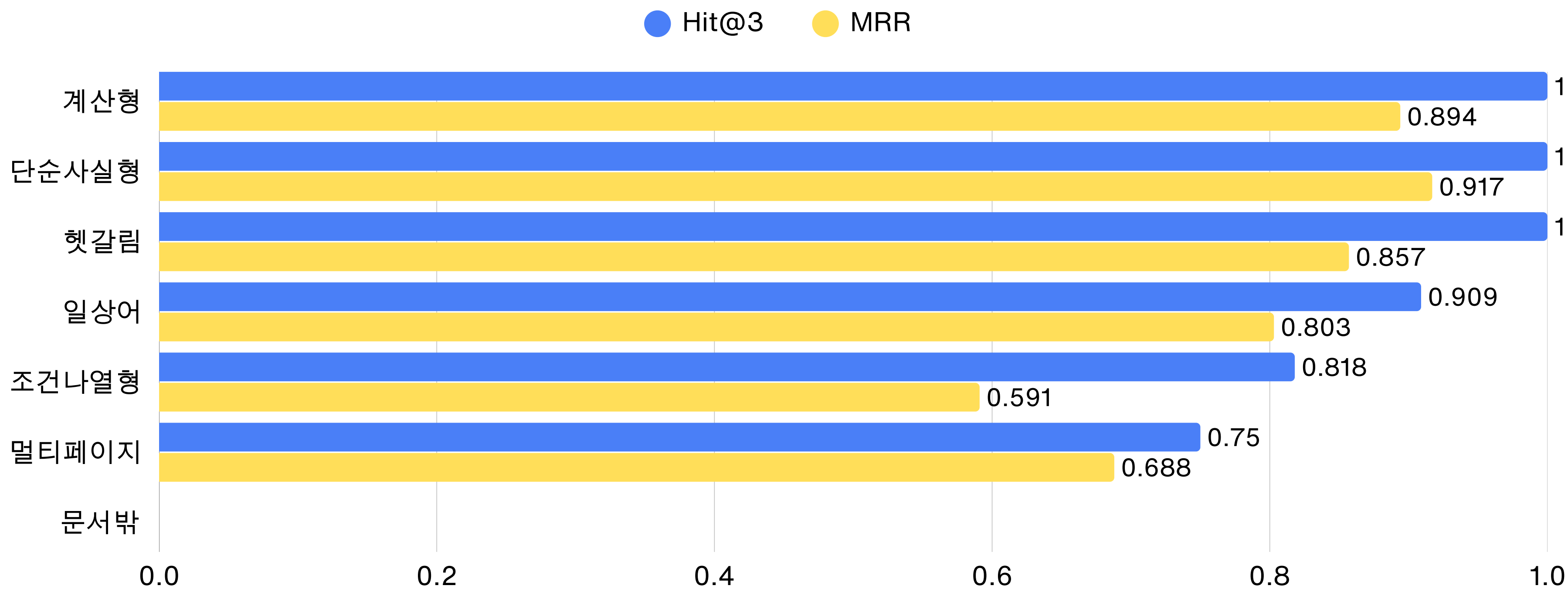
66문항 - Hit@3 / MRR



단순사실형·계산형에는 강했지만 조건나열형·멀티페이지 질문에서는 상대적으로 낮은 성능

질문 유형별 성능 (chunk 300 + rerank)

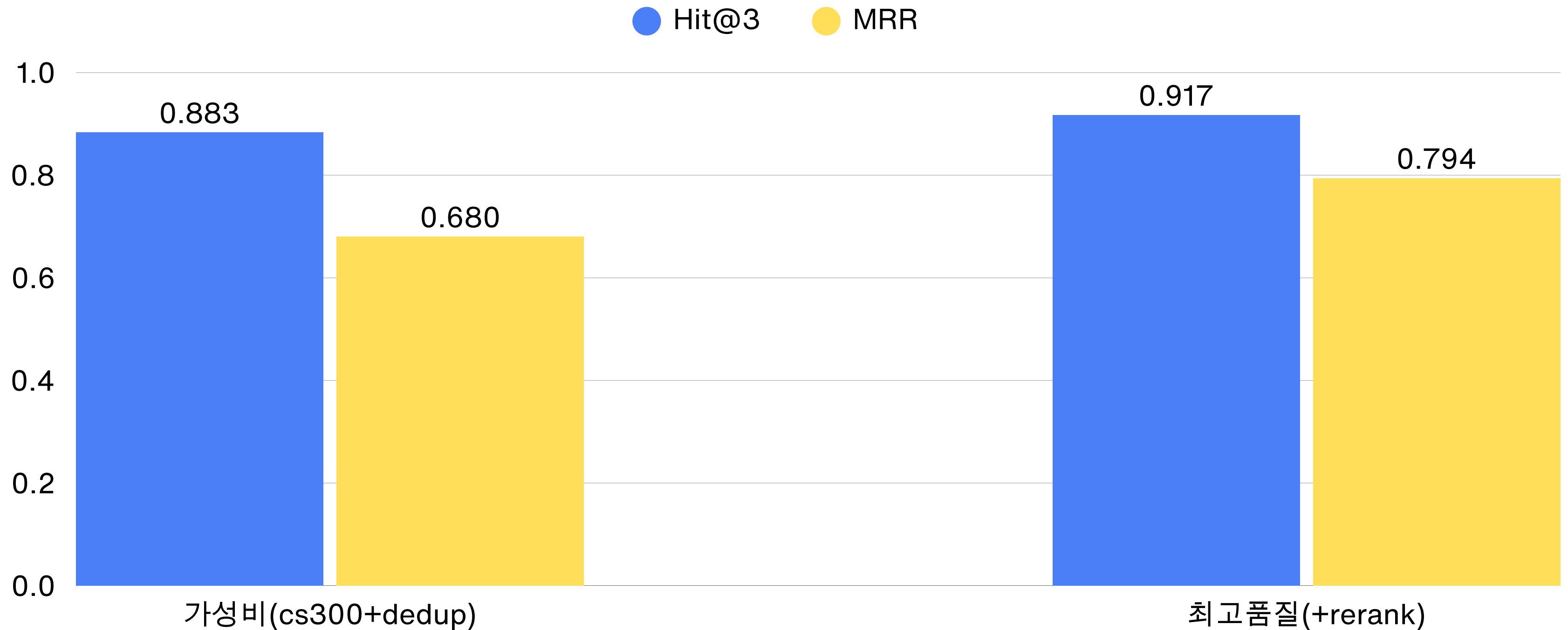
66문항 - Hit@3 / MRR



Reranker 적용 후 대부분 유형이 개선됐지만 **멀티페이지** Hit@3는 0.750에 머물러 추가 개선

최종 모델 구성 비교

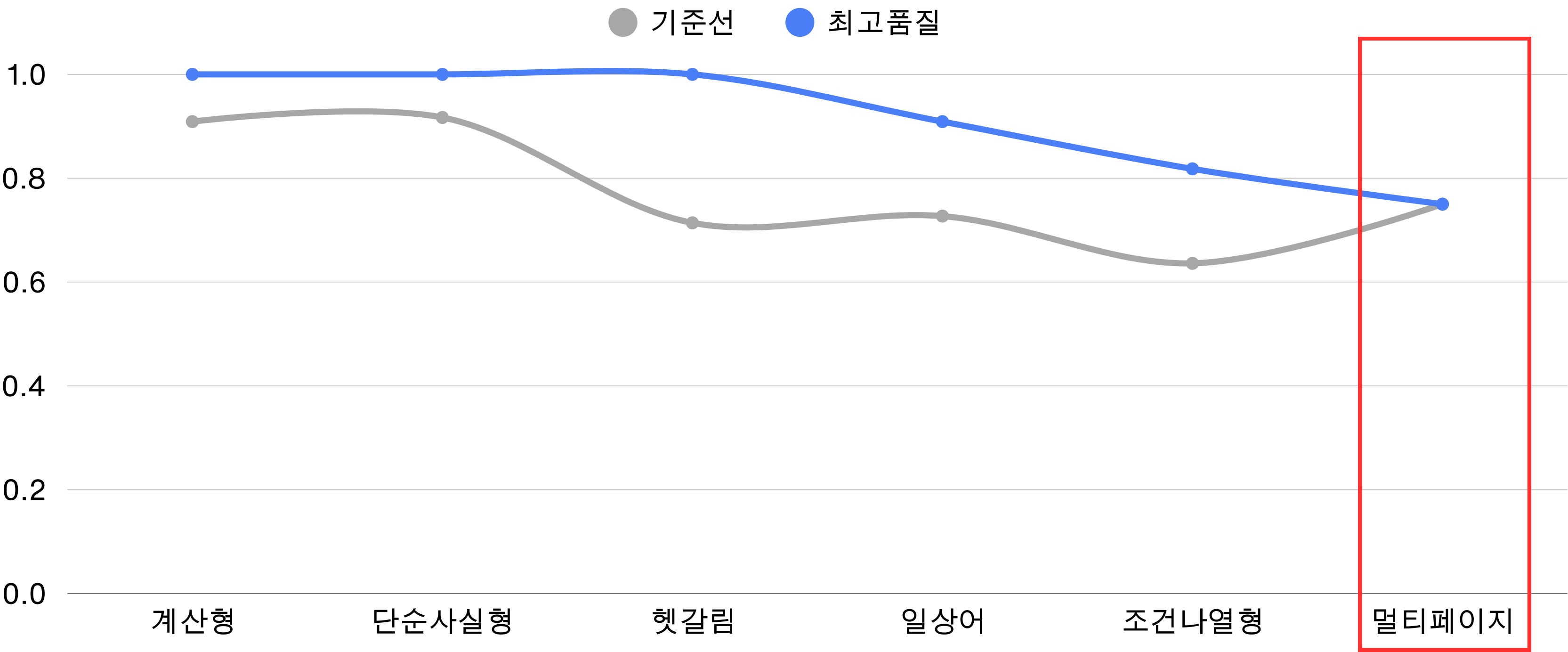
66문항 - Hit@3 / MRR



Reranker는 검색 성능을 높였지만 응답시간이 증가했으므로
Dedup은 운영 기본안(**가성비**), **Reranker**는 Recall 우선안(**최고 품질**)으로 분리

질문 유형별 개선 효과 (기준선 → 최고품질)

66문항 - Hit@3



대부분 유형이 개선됐지만 **멀티페이지** 유형은 기준선과 동일해 개선 효과가 없음

결론

전처리·평가 신뢰성 확보

머리말 제거와 복수 정답 기준으로 **검색 노이즈와 거짓 실패 교정**

최종 RAG 파이프라인 완성

chunk300 · bge-m3 · Similarity+Dedup · top_k3 · Upstage+Reasoning

13-->26→66문항 일반화 검증

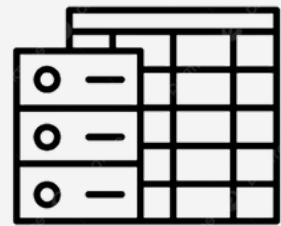
문항 수와 실패 유형을 단계적으로 확대하고, 기존 개선안을 최종 조건에서 재검증

목적별 운영 모드 분리

빠른 운영: Dedup — Hit@3 0.883 · 숫자 정확도 0.853 · **1.49초**

Recall 우선: Reranker — Hit@3 **0.917** · 숫자 정확도 0.812 · 5.72초

어려웠던 점



표(테이블) 인식

표를 그대로 텍스트로 뽑다 보니
세울표 같은 게 줄과 칸이 뒤섞여
납작하게 뭉개짐



특수기호(①~⑪) 깨짐

깨진 문자로 인덱싱되면 임베딩이
"①"이라는 의미 있는 기호가 아니라
무의미한 한글 조합으로 처리됨



평가셋 규모

과적합 우려로 **평가셋을 확장할 때**
마다 성능이 뚜렷하게 떨어짐



공통 실패 문항

조건·숫자·예외가
여러 문단에 흩어져 있는 항목에
대해서 **지속적으로 찾지 못함**

THANK YOU

